

- PJ2 模糊测试项目报告
 - 第一部分：测试目标
 - 第二部分：测试环境
 - 第三部分：测试对象
 - 第四部分：测试内容与命令
 - 4.1 单元测试
 - 4.2 单次运行测试
 - 4.3 批量对比测试
 - 4.4 正式实验套件测试
 - 第五部分：测试结果
 - 5.1 单元测试结果
 - 5.2 单次运行结果
 - 5.3 批量对比结果
 - 5.4 正式实验套件结果
 - 第六部分：关键截图建议
 - 第七部分：结论模板

PJ2 模糊测试项目报告

- 课程名称：软件质量保障与测试
- 实验名称：PJ2 - 模糊测试（Fuzzing）
- 小组编号：17
- 小组成员：王龄峰， 朱相成， 李子涵， 肖羽平， 洪昭晖
- 完成日期：2026-05-29

第一部分：测试目标

本测试报告用于说明本组在 PJ2 模糊测试项目中的运行环境、测试对象、测试命令、实验参数以及测试结果。测试重点包括：项目是否能够正常启动、不同调度策略是否能够正常运行、结果是否能够正确持久化，以及实验统计数据是否能够完成汇总输出。

第二部分：测试环境

测试环境如下：

项目	内容
操作系统	Windows-11-10.0.26200-SP0
Python 版本	Python 3.14.0
项目路径	simple_fuzzer/
主要依赖	Python 标准库
测试时间	2026-05-29

可以补一句：

本项目主要依赖 Python 标准库运行，不需要复杂的第三方环境配置，便于在统一参数下完成多轮实验。

第三部分：测试对象

建议写成：

本次测试对象包括以下几个部分：

- `main.py`：单次 fuzzing 运行入口
- `compare_schedules.py`：批量调度策略对比入口
- `run_experiment_suite.py`：正式实验套件运行入口
- `summarize_results.py`：结果汇总入口
- `view_result.py`：结果查看入口
- `simple_fuzzer/tests/`：单元测试集合

同时，本次测试覆盖的被测样例为：

- `sample1`
- `sample2`
- `sample3`
- `sample4`

第四部分：测试内容与命令

这一部分建议拆成 4 小节。

4.1 单元测试

测试命令：

```
python -m unittest discover -s simple_fuzzer/tests -p "test_*.py"
```

测试目的：

- 验证变异器功能是否正常
- 验证路径调度与新增调度策略是否正常
- 验证实验输出与汇总脚本是否正常

测试结果如下：

```
Ran 51 tests in 1.039s  
  
OK
```

4.2 单次运行测试

本次代表性单次运行使用正式实验脚本自动生成的单次结果，等价于对不同 `sample` 和 `schedule` 分别执行一次完整 fuzzing。对应入口命令如下：

```
python main.py --sample 1 --run-time 10 --schedule path --quiet --save-population
```

测试目的：

- 验证项目能否完成一次完整 fuzzing 流程
- 验证结果文件能否落盘
- 验证 `population` 快照能否按需保存

预期输出：

- `_result/single/*.pkl`
- `_result/single/*.json`

- `_result/single/*.txt`
- `_result/single/*-population.json` (可选)

4.3 批量对比测试

本次批量对比测试采用如下参数：

```
python compare_schedules.py --samples 1 2 3 4 --schedules uniform path rare --run-time 10 --repeats 3
```

测试目的：

- 验证多个样例和多个调度策略能否统一运行
- 验证原始结果与汇总结果是否能正确生成

预期输出：

- `_result/comparison/raw_results.json`
- `_result/comparison/raw_results.csv`
- `_result/comparison/summary.csv`
- `_result/comparison/summary.txt`

4.4 正式实验套件测试

本次正式实验实际使用的命令如下：

```
python run_experiment_suite.py --label official --samples 1 2 3 4 --schedules uniform path rare --run-time 10 --repeats 3 --capture-single-runs --save-population
```

测试目的：

- 验证正式实验计划能否被完整记录
- 验证实验矩阵能否按统一参数执行
- 验证实验目录是否自动生成
- 验证报告素材是否能自动汇总

预期输出：

```
_result/runs/official/  
├─ experiment_plan.json  
├─ comparison/  
├─ single/  
└─ report_assets/
```

第五部分：测试结果

5.1 单元测试结果

测试结果如下：

测试项	结果
变异器测试	通过
路径调度测试	通过
新增调度策略测试	通过

测试项	结果
实验输出测试	通过
汇总脚本测试	通过

简述模板：

单元测试结果表明，项目中的关键模块均能够在当前环境下正常运行，说明变异器、调度策略、结果输出和结果汇总逻辑整体正确。

5.2 单次运行结果

从 `runs/official/report_assets/single_runs.csv` 中选取代表性单次结果如下：

sample	schedule	run_time	initial_seed_count	covered_line_count	crash_count	total_paths	total_execs
1	path	10	2	8	6	5	7995
2	uniform	10	1	13	4	5	37936
3	rare	10	1	9	9	9	68011
4	uniform	10	4	232	0	401	3014

分析模板：

从单次运行结果来看，项目能够在指定时间内完成 fuzzing 循环，并输出覆盖行数、崩溃数量、路径数与执行次数等关键指标，说明单次运行流程完整有效。

5.3 批量对比结果

批量对比结果如下：

sample	schedule	runs	avg_execs	avg_paths	avg_covered_lines	avg_crashes
1	path	3	8230.333	5	8	6
1	rare	3	9013.667	5	8	6
1	uniform	3	9928.667	5	8	6
2	path	3	33466	5	13	4
2	rare	3	32454.333	5	13	4
2	uniform	3	34742.667	5	13	4
3	path	3	58017.333	9	9	8.667
3	rare	3	61543.667	9	9	9
3	uniform	3	56345.333	9	9	9
4	path	3	1673	275.667	225.333	0
4	rare	3	1305.333	233.333	225	0
4	uniform	3	2656	397.667	448	0

结果分析如下：

- 在 `sample1` 和 `sample2` 上，三种策略的覆盖行数和路径数基本一致，说明这些样例在 10 秒内较容易达到相近覆盖。
- 在 `sample3` 上，`rare` 和 `uniform` 的平均崩溃数略高于 `path`，说明新增策略在该样例上的异常触发能力略强。
- 在 `sample4` 上，`uniform` 的平均覆盖行数达到 448，明显高于 `path` 和 `rare`，同时平均路径数和执行次数也最高，说明该样例下基线调度策略表现更好。
- `sample4` 上没有发现崩溃，说明当前输入变异能够扩展覆盖范围，但尚未触发明显异常。

5.4 正式实验套件结果

本次正式实验套件运行结果如下：

- 实验计划是否成功生成
- 各目录产物是否完整
- 单次结果、对比结果和报告素材是否能够自动汇总

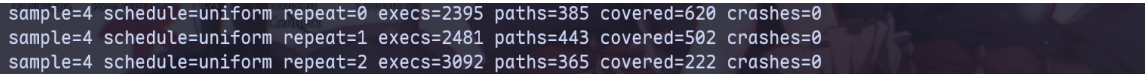
本次实验成功生成了 `experiment_plan.json`，并在 `runs/official/` 下自动输出了 `comparison/`、`single/` 和 `report_assets/` 三类结果目录。其中 `comparison/` 包含原始结果和汇总表，`single/` 包含代表性单次运行结果和 `population` 快照，`report_assets/` 则自动汇总了测试报告可直接使用的表格和摘要文件。这说明当前框架已经具备较完整的实验组织与交付支撑能力。

第六部分：关键截图建议

单元测试通过截图



run_experiment_suite.py 执行过程截图



report_assets/summary.md 或 summary.csv 截图

sample	sched...	runs	run_ti...	initial_seed_count	avg_e...	avg_p...	avg_covered_lines	avg_cras...	avg_duration_seconds		
1	path	3	10	2	8230.3...	5	8	6	10.001		
1	rare	3	10	2	9013.6...	5	8	6	10.026		
1	uniform	3	10	2	9928.6...	5	8	6	10.001		
2	path	3	10	1	33466	5	13	4	10.002		
2	rare	3	10	1	32454....	5	13	4	10.001		
2	uniform	3	10	1	34742....	5	13	4	10.001		
3	path	3	10	1	58017....	9	9	8.667	10.002		
3	rare	3	10	1	61543....	9	9	9	10.003		
3	uniform	3	10	1	56345....	9	9	9	10.002		
4	path	3	10	4	1673	275.667	225.333	0	10.007		
4	rare	3	10	4	1305.3...	233.333	225	0	10.006		
4	uniform	3	10	4	2656	397.667	448	0	10.003		

view_result.py 查看单次结果截图

Sample: 1
Schedule: path
Requested run time: 10
Initial seed count: 2
Total execs: 7995
Total paths: 5
Covered line count: 8
Crash count: 6
Duration seconds: 10.001

3. 测试环境

Crashes:

Covered lines:

Crashes:

Covered lines:

view_result.py

最简单的办法就是在终端里手动跑一两次 view_result.py，然后截终端输出。建议直接用你们正式实验里的一个结果。

bash

cd simple_fuzzer

python view_result.py _result

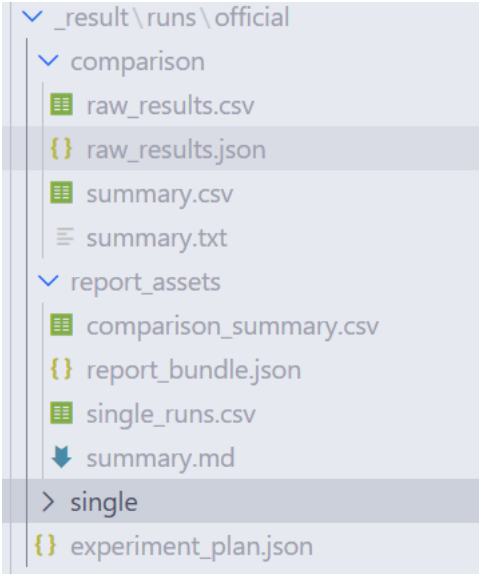
或者也可以看 pkl 版:

bash

cd simple_fuzzer

python view_result.py _result

_result/runs/official/ 目录结构截图



第七部分：结论模板

综合测试结果可以看出，本项目已经能够完成从单次 **fuzzing**、批量策略对比到正式实验编排的完整流程，并能够将结果以结构化形式输出到文件系统中。项目在当前环境下运行稳定，主要功能模块通过测试，能够满足实验说明中对代码实现、结果持久化、实验统计和测试材料整理的要求。

需要说明的是，实验结果仍会受到运行时间、初始种子数量和样例复杂度的影响，因此不同策略之间的差异还需要在更长时间或更多重复实验下进一步观察。